Faster matrix multiplication in Java with better loops and sparse matrices

Pablo Castillo Garcerá
Big Data, Individual Assignment 2, 2026-2027
GitHub: https://github.com/Pablo5250/matrix-multiplication (folder assignment2)

1. The problem

In the first assignment I multiplied two square matrices A and B of size n x n with the simplest method: three loops in the order i-j-k. Each value c[i][j] of the result C is the sum of n products, so the program does n x n x n multiplications. It worked, but it was slow with big matrices. Java needed 37 seconds for n = 2048.

The number of operations is only half of the problem. A big part of the time is lost waiting for the memory. The numbers of the matrices are saved in the RAM, which is big but slow compared with the CPU. To avoid waiting, the CPU has a cache, a small and very fast memory inside the CPU. When the program asks for a number, the CPU copies it from the RAM to the cache together with the numbers saved next to it. If the next number the program needs is already in the cache, it arrives almost at once. If it is not, the CPU stops and waits for the RAM, and that can take around 100 times longer. This is why the cache is so important for speed: a program that reads the memory in order can be many times faster than one that jumps around, even if both do the same maths. My CPU has three levels of cache. L1 is the smallest and fastest one, with 32 KB for each core. L2 has 512 KB for each core, and L3 has 32 MB shared by all the cores.

In this second assignment I use only Java and I try two ideas to go faster. In part A I keep dense matrices. A dense matrix is a normal matrix, where the computer saves every number, also the zeros. A dense matrix of 2048 x 2048 has about 4.2 million numbers, and each one uses 8 bytes, so it needs 32 MB. In this part I do not change the maths at all. I only change the order in which the program reads the numbers, so that it uses the cache better.

In part B I use sparse matrices. A sparse matrix is a matrix where almost all the numbers are zero. This is very common with real data. Imagine a table with all the users of Netflix in the rows and all the films in the columns, with the score that each user gave to each film. A normal user has seen only a few films, so almost every cell of the table is zero. The same happens with social networks (who follows who) or with the words that appear in each document. Saving and multiplying all those zeros wastes a lot of memory and time, and with millions of rows it is not even possible. For this reason sparse matrices are one of the basic tools of Big Data. The idea of part B is to save and multiply only the numbers that are not zero, which I call the nonzeros. To say how sparse a matrix is I use the density, the percentage of nonzeros. For example, with 0.1% density only one number in a thousand is a nonzero, and the other 99.9% are zeros.

I want to know how much each idea helps, and when the sparse idea stops being a good idea. Before measuring, I expected i-k-j to be much faster than the basic version, and blocking to be the best one for very big matrices (both are explained in the next section). For part B, I thought the sparse version would win easily when there are very few nonzeros, and lose when the matrix has many of them, because finding where each number is costs extra work.

2. The methods I compared

The baseline is the same Java code as in Assignment 1. To get each value c[i][j], it walks along row i of A and column j of B. The column is the problem. Java saves a matrix row by row, so the numbers of one row are next

to each other in memory, but two numbers of the same column are n numbers apart. When the loop goes down a column of B, almost every number is a new trip to the RAM.

My first optimisation, i-k-j, only swaps the two inner loops. Now the inner loop takes one number a[i][k], multiplies it by the whole row k of B and adds the results to row i of C. Rows are saved one number after the other, so the CPU reads the memory in order. The operations are the same and the result is exactly the same.

The second optimisation is blocking, also called tiling. It does the same i-k-j work, but in small squares of 64 x 64 numbers called tiles. A tile of 64 x 64 doubles uses 32 KB, the size of the L1 cache, so while the program works on a tile, its numbers stay in the cache and are used many times. I chose 64 before measuring, and later I also tried other sizes.

For part B I need a way to save only the nonzeros. I use CSR (Compressed Sparse Row), one of the most common formats for sparse matrices. CSR keeps three lists. The first one has the nonzero values, row after row. The second one has the column of each value, and the last one says where each row starts in the first list. For example, the 4 x 4 matrix with rows (0 5 0 0), (0 0 8 0), (0 0 0 3) and (1 0 0 0) is saved as values = [5, 8, 3, 1], columns = [1, 2, 3, 0] and row starts = [0, 1, 2, 3, 4]. The 12 zeros are not saved at all. Each nonzero needs 12 bytes, 8 for the value and 4 for its column, so CSR only saves memory when there are few nonzeros.

To multiply two CSR matrices I use the method of Gustavson. For each row i of A, I take every nonzero a[i][k], multiply it by the nonzeros of row k of B, and add the results into a work row. Then the nonzeros of that work row become row i of C. The program never touches a zero, so the work depends on the number of nonzeros and not on n x n x n. I wrote all this code myself, without any library.

3. How I measured

All the methods use the same matrices, so the comparison is fair. I make them in Java with random numbers between 0 and 1 and a fixed seed (the starting number of the random generator), so every run gets exactly the same matrices. In the sparse ones, each position has a chance equal to the density of getting a number, so the nonzeros can be anywhere. In part B the dense method receives the same sparse matrices, converted to normal matrices before the timer starts.

Before measuring I checked the results with 63 tests: small products that I did by hand, the identity matrix (ones on the diagonal and zeros in the rest), the zero matrix, and sizes that are not a multiple of 64. The two optimised methods give exactly the same numbers as the baseline, and CSR gives exactly the same numbers as the dense method. In the big runs I also compared the sum of all the values of C: CSR and dense agreed in the 11 cases where both were run. The baseline is the code that I already checked against NumPy in Assignment 1.

The timer only measures the multiplication, including creating the result matrix. Creating the inputs is not measured. Converting a dense matrix to CSR has its own timer, because in real life the data sometimes arrives as a dense matrix.

Java needs some care when measuring time. Java code runs inside the JVM (Java Virtual Machine), the program that runs Java code. At the start the JVM does not run fast machine code. While the program runs, the JIT compiler (just-in-time compiler) finds the parts that are used many times and translates them into machine code. So the first runs of a method are slower than the next ones. For this reason each test runs in a new JVM, with 3 warm-up runs that I do not count and then 5 measured runs. I report the median, which is the middle value of the 5 runs, and the IQR (interquartile range), which tells how different the runs were. A small IQR means stable times. Everything runs on one core, which is one of the 12 processing units of my CPU.

I also decided some limits before starting: 20 minutes and 8 GB of memory for each test. When a test was predicted to need more, I skipped it and wrote it down. For memory I give two numbers. One is the maximum RAM used by the Java process, read with psutil, a Python library that reads information about the programs

that are running. The other one is the size of only the data: 8 bytes per number for dense matrices, and 12 bytes per nonzero for CSR, because each value also needs its column.

The computer is an AMD Ryzen 9 5900X with 32 GB of RAM and Windows 11. I used OpenJDK 25.0.1. The heap is the part of the memory where Java keeps the objects of the program, like the matrices, and I let it grow up to 8 GB (option -Xmx8g).

4. Results of part A

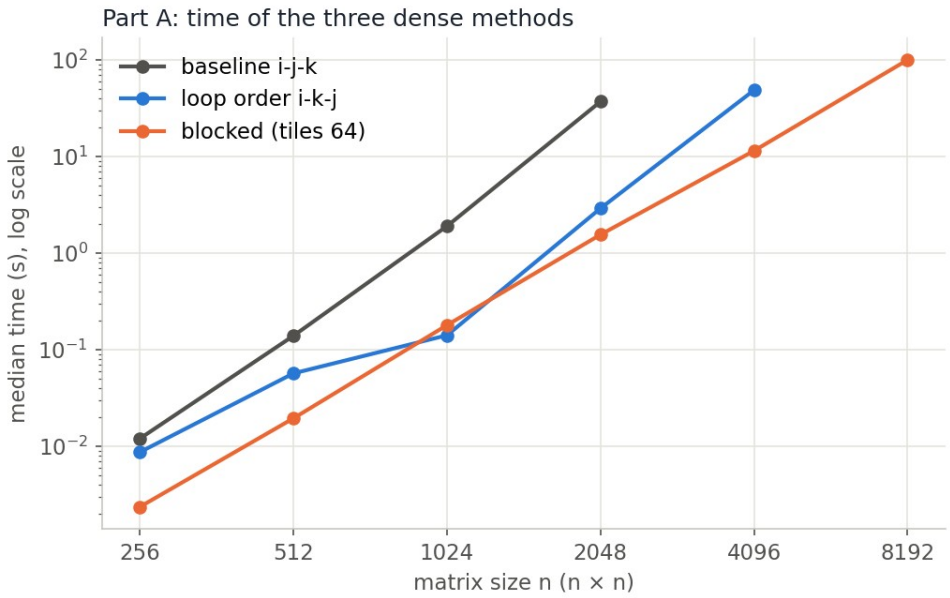


Figure 1. Time of the three dense methods. Both axes use a logarithmic scale, where each step is 10 times more.

The baseline is always the slowest. At n = 2048 it needs 37 seconds, and n = 4096 was skipped because the prediction was about 40 minutes. The i-k-j version starts very well, with only 0.14 s at n = 1024, but after that it gets much worse: 48.8 s at n = 4096. Blocking grows in a smooth way, and it was the only method that finished n = 8192 (in 100 seconds).

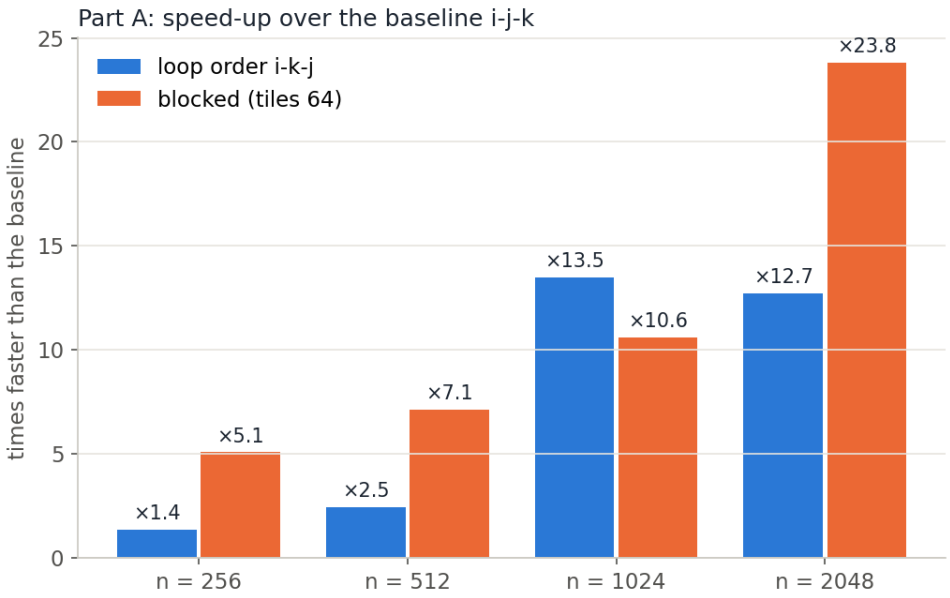


Figure 2. How many times faster each optimised method is than the baseline at the same size.

At n = 2048 blocking is 23.8 times faster than the baseline, and i-k-j is 12.7 times faster. With small matrices the gain is much smaller, for example only 1.4 times for i-k-j at n = 256. At that size all the matrices fit in the cache anyway, so the order of reading does not change much.

I think the size of B explains why i-k-j slows down. For every row of C, i-k-j reads the whole matrix B. At n = 2048 B needs 32 MB, the same as the L3 cache, so B cannot stay there and the CPU brings it again from the RAM for every row. Blocking only needs a few small tiles at the same time. At n = 4096 it was 4.2 times faster than i-k-j.

Table 1. Part A. The column times faster compares each method with the baseline at the same n. 1 MiB is 1,048,576 bytes, almost the same as 1 MB.

Method	n	Runs	Median time	IQR	Times faster	Peak memory (MiB)
baseline i-j-k	256	5	12.1 ms	0.120 ms	1.0	53
baseline i-j-k	512	5	140.2 ms	4.2 ms	1.0	71
baseline i-j-k	1024	5	1.91 s	32.3 ms	1.0	118
baseline i-j-k	2048	5	37.17 s	1.37 s	1.0	302
baseline i-j-k	4096	-	skipped: over the 20 min limit	-	-	-
baseline i-j-k	8192	-	skipped: over the 20 min limit	-	-	-
i-k-j	256	5	8.8 ms	0.204 ms	1.4	55
i-k-j	512	5	57.2 ms	0.390 ms	2.5	71
i-k-j	1024	5	141.8 ms	21.4 ms	13.5	120
i-k-j	2048	5	2.92 s	251.2 ms	12.7	302
i-k-j	4096	5	48.76 s	4.11 s	-	997
i-k-j	8192	-	skipped: over the 20 min limit	-	-	-
blocked (tile 64)	256	5	2.4 ms	0.027 ms	5.1	54
blocked (tile 64)	512	5	19.6 ms	0.031 ms	7.1	71
blocked (tile 64)	1024	5	180.5 ms	20.8 ms	10.6	120
blocked (tile 64)	2048	5	1.56 s	69.2 ms	23.8	303
blocked (tile 64)	4096	5	11.50 s	494.4 ms	-	997
blocked (tile 64)	8192	5	100.31 s	2.99 s	-	3,378

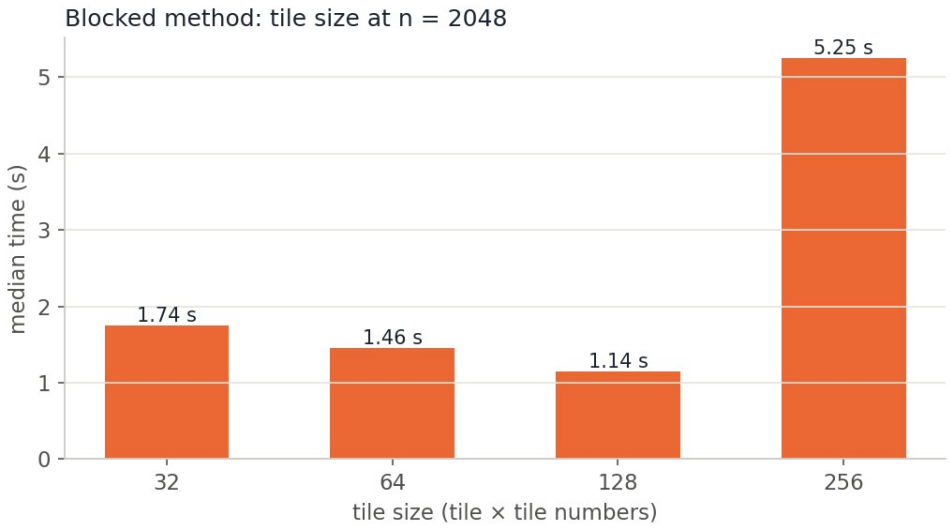


Figure 3. Blocking with different tile sizes at n = 2048.

My first choice of 64 was not the best one. Tiles of 128 were a bit faster (1.14 s against 1.46 s), but tiles of 256 were much slower (5.25 s). Three tiles of 256 x 256 numbers need 1.5 MB, more than the 512 KB of the L2 cache. So the best tile size depends on the cache, and on another computer it could be different.

5. Results of part B

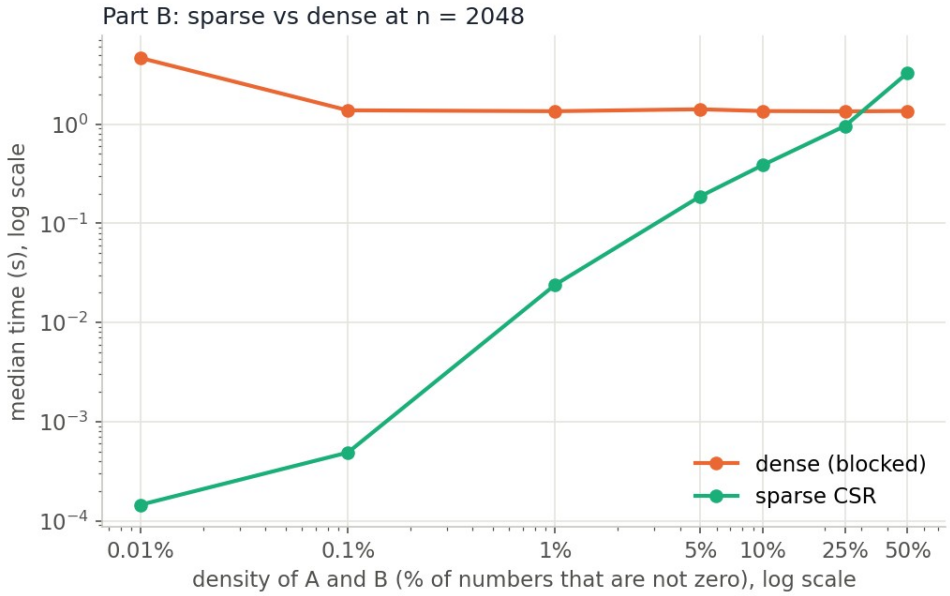


Figure 4. Sparse (CSR) and dense methods at n = 2048 for different densities. Both axes use a logarithmic scale.

The dense line is flat, about 1.35 s for every density, because the dense method multiplies all the numbers, zeros included. The CSR line goes up with the density. With 0.1% nonzeros CSR needs only 0.5 ms, around 2,800 times less than dense. At 25% CSR is still a bit faster (0.96 s against 1.35 s), but at 50% it is clearly slower (3.28 s against 1.36 s).

The first dense point, at 0.01%, looks strange: 4.6 s. The density is not the reason. In that Java process the JIT compiler did not finish optimising the code during the warm-up, so all the runs stayed slow. I did not delete this result. I repeated the test two more times and both times the time went down to 1.36 s after the second run (file results/raw/recheck_dense_n2048_d0.01.csv).

Table 2. Part B at n = 2048. A and B have the same density. The last column is the time to convert A from dense to CSR, measured on its own.

Density	Nonzeros in A	Nonzeros in C	Density of C	CSR time	Dense time	Convert to CSR
0.01%	409	77	0.00184%	0.145 ms	4.65 s	2.7 ms
0.1%	4,063	8,104	0.193%	0.488 ms	1.38 s	2.5 ms
1%	41,817	776,249	18.5%	24.0 ms	1.35 s	3.0 ms
5%	209,808	4,169,589	99.4%	186.7 ms	1.41 s	5.2 ms
10%	419,323	4,194,304	100%	385.3 ms	1.36 s	7.4 ms
25%	1,047,258	4,194,304	100%	959.0 ms	1.34 s	14.9 ms
50%	2,095,225	4,194,304	100%	3.28 s	1.36 s	27.4 ms

Something that surprised me is how fast the result fills up. With 1% nonzeros in A and B, the result C already has 18.5% nonzeros, and with 5% it is almost full (99.4%). Each value of C is a sum of n products, and one product that is not zero is enough. For CSR this is bad news: when C is almost full, CSR has to write millions of values with their columns, and it also has to sort the columns of every row.

Memory goes in the same direction. At 50% density the CSR data of A, B and C needs 96 MiB, the same as the dense data, because every value carries 4 more bytes for its column. Moreover, converting a dense matrix to CSR costs time, between 2.5 and 27 ms at n = 2048. Next to the 1.35 s of the dense method that is nothing, but at 0.1% it is 5 times more than the CSR multiplication itself (2.5 ms against 0.5 ms). So if the data comes as a dense matrix, most of the time goes into the conversion.

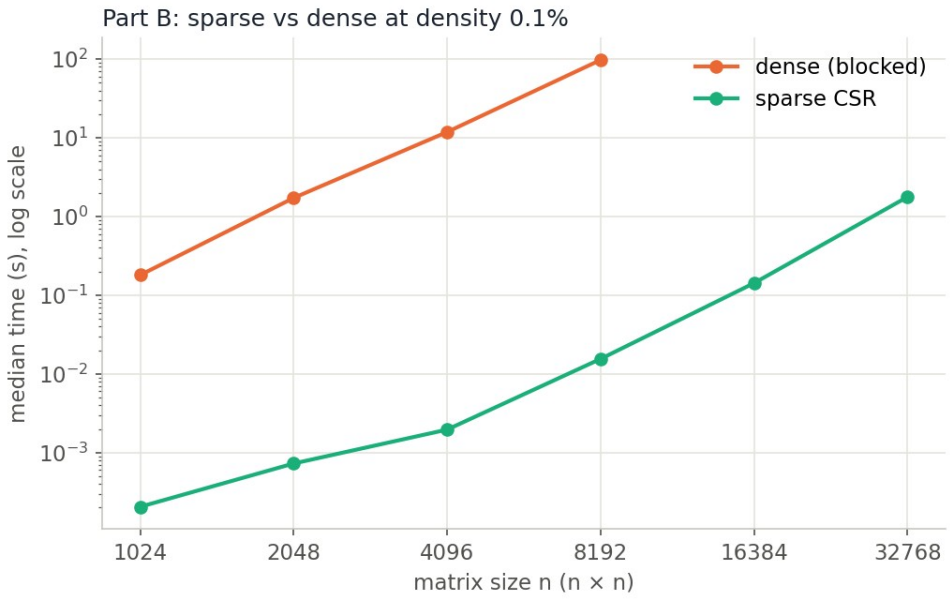


Figure 5. Sparse (CSR) and dense methods at 0.1% density for bigger and bigger n.

With big and sparse matrices the difference is enormous. At 0.1% density and n = 8192, CSR needs 0.016 s and the dense method needs 98 s, more than 6,000 times more. The dense method stopped at n = 8192, because n = 16384 would need almost 2 hours, while CSR finished n = 32768 in 1.8 s. The result fills up here too: from 0.1% in the inputs to 3.2% at n = 32768, because each row of C collects products from more rows of B.

Table 3. Part B at 0.1% density. Peak is the maximum memory of the Java process.

n	Nonzeros in A	Nonzeros in C	Density of C	CSR time	Dense time	CSR peak (MiB)	Dense peak (MiB)
1024	1,065	1,068	0.102%	0.205 ms	180.6 ms	59	118
2048	4,063	8,104	0.193%	0.727 ms	1.72 s	85	303
4096	16,754	67,400	0.402%	2.0 ms	11.86 s	190	1,130
8192	66,831	544,343	0.811%	15.5 ms	97.62 s	235	3,817
16384	267,664	4,352,110	1.62%	142.5 ms	skipped	386	-
32768	1,075,191	34,649,297	3.23%	1.77 s	skipped	8,117	-

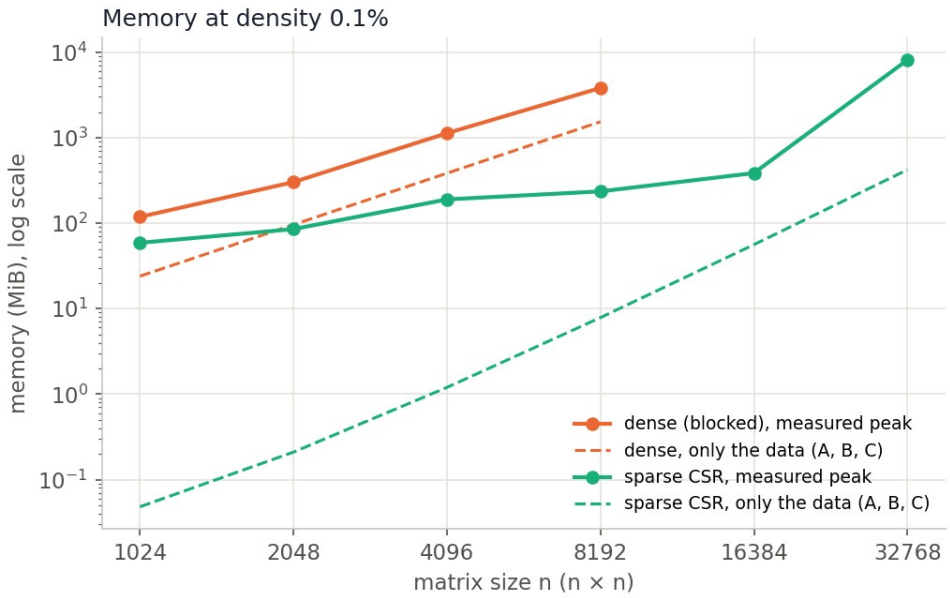


Figure 6. Memory at 0.1% density. Solid lines: maximum memory of the process. Dashed lines: size of only the data.

If we look only at the data, the difference in memory is huge: at n = 8192 the dense matrices need 1.5 GB and the CSR matrices only 8 MB. The measured peaks are much higher than the data. Java keeps old results in memory until the garbage collector (the part of Java that frees memory that is not used any more) cleans them, and with 8 GB available it waits a long time. That is why CSR at n = 32768 used 7.9 GB, very close to my limit, although its data was only about 0.4 GB. In Java the memory you measure depends a lot on the settings of the JVM.

6. What I learned

Part A surprised me because of how much a small change can do. The three dense methods do the same operations, but changing the order of two loops made the program 12.7 times faster, and blocking made it 23.8 times faster at n = 2048. The whole gain comes from reading the memory in a better order.

In part B the gain comes from a different place: CSR does less work because it skips the zeros. With very few nonzeros it is thousands of times faster, it needs much less memory, and it can work with sizes that are impossible for dense matrices. However, the advantage disappears quickly when the density grows, and at 50% the dense method wins.

So the percentage of zeros alone is not enough to choose a method. The size of the matrix matters too. How full the result becomes is also important, especially if the data has to be converted to CSR first.

7. Problems and limits

I used only one computer, so the numbers can change on another machine, especially the best tile size, which I only tested at n = 2048. The nonzeros were placed at random. Real sparse matrices usually have some structure, for example numbers close to the diagonal, and CSR could behave differently with them. Java also adds noise: the JIT compiler changed one of my results, and the garbage collector makes the measured memory much bigger than the real data.

8. Next steps

Now everything runs on one of the 12 cores of the CPU. In Assignment 3 I will split the work between the cores. It would also be interesting to repeat part A with tiles of 128, and to try a sparse matrix multiplied by a dense one, which is common in machine learning.

References

T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed. MIT Press, 2022.

U. Drepper, "What every programmer should know about memory," Red Hat, Inc., 2007.

J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th ed. Morgan Kaufmann, 2017.

Big Data course notes, "Experiments: Monitoring and Performance Engineering," Grado en Ciencia e Ingeniería de Datos, 2026.

Y. Saad, Iterative Methods for Sparse Linear Systems, 2nd ed. SIAM, 2003.

F. G. Gustavson, "Two fast algorithms for sparse matrices: multiplication and permuted transposition," ACM Transactions on Mathematical Software, vol. 4, no. 3, pp. 250-269, 1978.

AMD, "Software Optimization Guide for AMD Family 19h Processors," pub. 56665, 2020.

Code and data: everything is in https://github.com/Pablo5250/matrix-multiplication, in the folder assignment2. There you can find the Java code, the tests, config.json with all the settings, the raw

measurements (results/raw/times.csv) and the scripts that make the tables and figures. run_all.bat repeats the whole experiment. The baseline is the Java code of Assignment 1.

Use of generative AI: I used Claude (Anthropic) as a support tool to help write the code and draft the text of this report. I reviewed all the code and the text, ran every test and measurement on my own computer, and checked the results. I am responsible for the code, the data and the conclusions.